Rogue Device Detection Using Passive Network Traffic: A Behavior-Driven, Self-Healing Approach with Per-Device Risk Scoring

Venkata Vasudeva Reddy Yeddula
Dept. of Computer Science & Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Andhra Pradesh, India
2320030400cse@gmail.com

Shivarathri Pradeep
Dept. of Computer Science & Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Andhra Pradesh, India
Roll No. 2320030412

Dr. K. Swanthana
Associate Professor
Dept. of Computer Science & Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Andhra Pradesh, India

***Abstract*—Home and campus local-area networks now host laptops, phones, and IoT devices, and a single compromised or rogue device can operate unnoticed for long periods. Signature-based intrusion detection and static firewall rules recognize only known patterns, overlook gradual behavioral drift, and require manual intervention once a threat is flagged. This paper presents a behavior-driven, self-healing network security system built on passive monitoring. A per-device behavioral baseline over packet rate, unique destination addresses and ports, DNS queries, and connection counts is learned online with Welford's algorithm, and a rule-based 0–100 risk score expresses each device's deviation, attaching a human-readable reason to every point added. When the score crosses a threshold, the device is isolated, then re-evaluated and restored once its behavior normalizes. In a reproducible 90-second simulation, an injected rogue device reached the maximum risk score of 100 and triggered an isolation action (logged in dry-run mode), while three known devices mostly remained in the normal range; all eight unit tests pass. A short baseline window produced some false positives on a legitimate device, motivating a robust estimator and a machine-learning anomaly layer. No accuracy, precision, or recall is claimed yet; these are left to repeated evaluation on a live testbed.**

***Keywords*—rogue device detection, passive monitoring, behavioral baseline, risk scoring, self-healing network, IoT security, Welford's algorithm**

I. Introduction

Modern home and campus local-area networks (LANs) host a heterogeneous mix of laptops, smartphones, and Internet-of-Things (IoT) devices. In such an environment a single compromised or rogue device can scan the network, exfiltrate data, or join a botnet [1] long before an administrator notices. Conventional defenses—firewalls with static rules and signature-based intrusion detection systems (IDS)—recognize only attacks that match a known pattern. They miss novel behavior and slow behavioral drift, do not separate a mildly unusual device from a dangerous one, and typically need manual intervention once a threat is flagged, leaving a window in which the device keeps operating.

A large body of recent work shows that devices can be identified passively from traffic timing, behavioral phases, DNS metadata, or service-level flows [2], [3], [4], [5], [6]. Separately, self-healing frameworks for software-defined networks (SDN) and cloud environments automate detection and recovery, but target infrastructure and controller faults rather than the behavior of individual endpoints [7], [8]. Among the sources surveyed for this work, none fuses continuous per-device behavioral risk scoring with automated isolation and recovery.

This paper closes that gap with a lightweight, interpretable prototype. Its contributions are: (1) a passive monitoring pipeline that learns a per-device behavioral baseline online using Welford's algorithm [9]; (2) a rule-based 0–100 risk score that explains every point it assigns; (3) an isolate–re-evaluate–restore response loop, run in dry-run mode by default; and (4) a live dashboard and an honest preliminary evaluation, including its limitations.

The remainder of the paper is organized as follows. Section II reviews related work and the research gap. Section III describes the proposed system. Section IV gives the implementation and experimental setup. Section V presents results, Section VI discusses limitations, and Section VII concludes.

II. Related Work

A. Passive Device Fingerprinting

Neumann et al. identify wireless devices from global traffic parameters in 802.11 captures and find that inter-arrival time is the strongest passive signal [2]. Prasad et al. model phase-based behavioral patterns to fingerprint IoT devices behind a gateway [3], and later work shows that identification is feasible from very short observation windows [4] and from passive DNS traffic alone, without payload inspection [5]. Service-level (port-based) fingerprints, evaluated on roughly ten million IPFIX flow records from 13 device types, offer a lightweight and explainable alternative to per-packet machine learning [6]. All of these stop at identification or classification; none quantifies how dangerous a device's current behavior is.

B. Behavioral Anomaly Detection and Risk Scoring

N-BaIoT trains deep autoencoders on 115 statistical features to detect IoT botnet traffic, but it works on a fixed

offline dataset and outputs a binary or multi-class label [1]. Rule-based mismatch checks over device attributes flag anomalies without machine-learning overhead, yet they raise only a binary flag [10]. A patent on fingerprint merging shows that dynamic, feedback-updated risk metrics are feasible, but in an enterprise setting rather than an open academic LAN framework [11].

C. Self-Healing Networks and Automated Response

Surveys report that automated isolation and recovery shortens manual response time [12], and AI-assisted SDN schemes isolate malicious flows while preserving legitimate traffic [13]. A taxonomy of SDN security automation observes that self-healing solutions remain less mature than reactive detection [14]. GeckoNet detects, isolates, and recovers SDN faults on an emulated topology [8], and SecuNet-4D combines graph-neural-network anomaly detection with self-healing failover [7]. These systems operate at the infrastructure or controller level, not on per-device behavioral baselines.

D. Research Gap

Table I summarizes the gap. Fingerprinting work identifies devices, rule-based analytics raise binary flags, and self-healing work protects infrastructure; the combination of a graded per-device risk score with automated containment and recovery is not covered.

TABLE I
SUMMARY OF RELATED APPROACHES AND GAPS

Approach	Representative work	Gap
Passive / behavioral fingerprinting	[2], [3], [4], [5], [6]	Identification only; no risk score or response
Deep-learning IoT detection	[1]	Offline, global model; label output, no response
Rule-based mismatch analytics	[10]	Binary flag; no graded 0–100 score
Self-healing SDN / cloud	[12], [13], [14], [8], [7]	Infrastructure-level; no per-device baseline
Risk-metric patent	[11]	Enterprise scope; not an open LAN framework

III. PROPOSED SYSTEM

A. Architecture

The system is a five-stage pipeline (Fig. 1). Passive capture observes traffic to and from every LAN device without sending or probing anything. A feature extractor aggregates events into per-device, per-window vectors. A baseline profiler learns each device's normal behavior, a rule-based risk engine scores deviation, and a response module isolates and later restores devices. Scores and alerts are persisted in SQLite and shown on a live Flask dashboard.

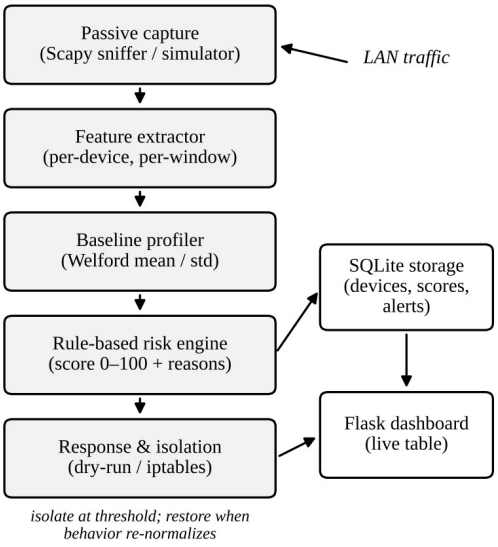


Fig. 1. System architecture of the proposed passive, self-healing rogue-device detector.

B. Passive Capture and Feature Extraction

Two interchangeable capture backends share one interface: a Scapy-based sniffer for a mirrored or monitor-mode interface on a testbed LAN, and a synthetic traffic simulator that needs no root privileges or network interface. For each device and time window the extractor computes five features: packet rate, number of unique destination IPs, number of unique destination ports, DNS query count, and connection count.

C. Online Baseline Learning

For each device and feature f, a running mean and variance are maintained with Welford's numerically stable online algorithm [9]. Given the n-th observation x:

$$\delta = x - \mu_{n-1}, \quad \mu_n = \mu_{n-1} + \delta/n \tag{1}$$

$$M_{2,n} = M_{2,n-1} + \delta(x - \mu_n), \quad \sigma_n = \sqrt{(M_{2,n}/(n-1))} \tag{2}$$

A device's baseline is considered established after a configurable number of learning windows ($W = 6$ in our experiments). Because the update is incremental, no history is stored and the baseline adapts online without offline retraining.

D. Rule-Based Risk Scoring

Once a baseline exists, each live feature value is converted to a deviation $z_f = (x_f - \mu_f)/\sigma_f$. The device's risk score is

$$S = \min(100, P_{\text{dev}} + P_{\text{allow}} + P_{\text{scan}}) \tag{3}$$

where P_{dev} sums per-feature penalties that grow with z_f; P_{allow} (+35 in our run) penalizes a device absent from the known-device allowlist; and P_{scan} (+25) fires on a port-scan-like pattern, i.e., when the number of unique destination ports reaches a configured threshold. Every contribution is logged with a human-readable reason, so an analyst can see why a device scored as it did. Scores map to four statuses—

normal, suspicious, high_risk, and critical—using thresholds read from a YAML configuration file rather than hard-coded.

E. Automated Response

When a device's score crosses the configured isolation threshold, the response module isolates it. It then re-evaluates the device periodically and restores access once behavior normalizes, closing the self-healing loop. By default the action is only logged (dry-run); real enforcement through iptables rules is opt-in via an *--enforce* flag, which keeps experiments safe on shared networks.

IV. Implementation and Experimental Setup

The prototype is written in Python 3 with a modular package layout (capture, features, baseline, scoring, response, storage, dashboard) so each stage can be tested and replaced independently. Scapy provides live sniffing, SQLite stores devices, scores, and alerts, Flask serves the auto-refreshing dashboard, and pytest drives eight unit tests covering per-device feature aggregation, packet-rate computation, empty windows, and the risk engine (normal traffic scores low, unknown devices are penalized, port-scan patterns reach high or critical status, and scores are clipped to 100).

Experiments use the simulator in a 90-second run (*python -m app.main --mode simulate --duration 90*). The simulated LAN has three known devices—a team laptop (AA:BB:CC:00:00:01), a phone (…:02), and an IoT camera (…:03)—each with characteristic normal traffic. About 15 seconds into the run, one rogue device (DE:AD:BE:EF:00:99, not in the allowlist) is injected and behaves like a port scanner issuing excessive DNS lookups. The baseline needs $W = 6$ windows, and isolation runs in dry-run mode. Dashboard output was checked against the console log and the SQLite database for the same run.

V. Results

Table II lists the score ranges observed for each device. The injected rogue device ranged from 60 to 100 and reached high_risk and critical status, whereas the known laptop, phone, and camera mostly stayed in the normal range, with the camera's low, steady traffic yielding the lowest scores.

TABLE II
Risk Scores Observed in the 90-Second Simulation

Device	Role	Score	Status
Team laptop (…:01)	Known, browsing	0–60	Normal / suspicious
Team phone (…:02)	Known, browsing	0–55	Normal / occ. suspicious
IoT camera (…:03)	Known, low traffic	0–16	Normal
Rogue (DE:AD: …:99)	Unknown, scan + DNS flood	60–100	High risk / critical, isolated

For the fully baselined rogue device the log shows the score reaching 100 with these reasons: packet_rate +18.5, unique_dst_ips +15.3, unique_dst_ports +16.1, dns_query_count +13.4, not in allowlist +35, and port-scan pattern +25. The contributions sum to 123.3 and are clipped to 100 (Fig. 2); the resulting action was "isolate mac=DE:AD:BE:EF:00:99 ip=192.168.1.199 (dry-run)". The 60-point lower bound of the rogue range equals the allowlist and port-scan penalties combined (35 + 25), consistent with an unknown scanner being flagged high-risk even before behavioral deviation accumulates.

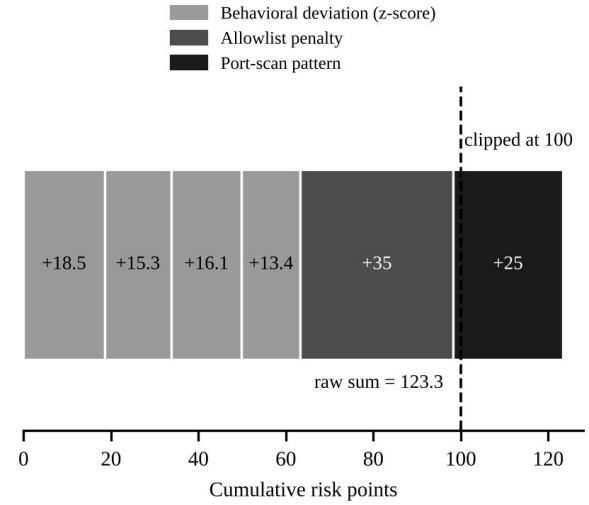


Fig. 2. Contributions to the rogue device's risk score; the raw sum of 123.3 is clipped to 100.

Table III compares the design with N-BaIoT [1]. The comparison is qualitative: the two systems were not benchmarked on a common dataset.

TABLE III
Qualitative Comparison with N-BaIoT

Aspect	N-BaIoT [1]	This work
Detection	Deep autoencoder, 115 features	Lightweight rules, interpretable features
Output	Binary / 10-class label	Continuous 0–100 score + reasons
Data	Fixed offline dataset	Live passive capture or simulator
Response	Stops at classification	Automated isolate / restore
Baseline	Global model	Per-device Welford mean / std
Adaptation	Offline retraining	Baseline adapts online

VI. Discussion

The results show that the rule-based score separates an injected scanner from ordinary devices and that every point of the score is explainable, which is the main advantage over opaque learned models. They are, however, preliminary, and several limitations apply.

Baseline noise. With only six learning windows the per-feature standard deviation is a noisy estimate, so the simulated laptop sometimes reached suspicious or high-risk status on ordinary variance—a false positive. Longer

learning windows or a robust estimator such as the median absolute deviation (MAD) should stabilize the score.

Evaluation scope. Traffic is synthetic, the run contains a single injected rogue device, and isolation was exercised only in dry-run mode. Accordingly, no accuracy, precision, recall, F1, or detection latency is claimed; repeated attack and benign runs are needed before such figures can be reported.

Design considerations. Because the baseline adapts online, a very slow drift could be absorbed into it; freezing or slowing baseline updates while a device's score is elevated is a natural safeguard. Allowlist penalties also assume device identities are not spoofed.

VII. Conclusion

This paper presented a working end-to-end prototype for rogue device detection: passive capture, per-device Welford baselining, interpretable 0–100 risk scoring, an isolate–re-evaluate–restore loop, and a live dashboard. In a reproducible simulation the injected rogue device reached the critical score of 100 while known devices mostly stayed normal, and all eight unit tests pass. Baseline noise is the main limitation.

Future work includes running the live capture module on a controlled LAN with a mirrored or monitor-mode interface, adding a MAD-based robust estimator and more learning windows, layering a machine-learning anomaly score beside the rule engine, enabling real iptables enforcement, and reporting detection rate, false-positive rate, and latency over repeated runs.

Acknowledgment

The authors thank their guide, Dr. K. Swanthana, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, for her guidance throughout this Engineering Capstone Project.

References

[1] Y. Meidan *et al.*, "N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jul.–Sep. 2018.

[2] C. Neumann, O. Heen, and S. Onno, "An empirical study of passive 802.11 device fingerprinting," arXiv:1404.6457, 2014.

[3] A. Prasad, K. Biju, S. Somani, and B. Mitra, "Context-aware behavioral fingerprinting of IoT devices via network traffic analysis," in *Proc. SECRYPT*, 2023, pp. 335–344.

[4] "Early-stage IoT device identification using passive network traffic analysis," arXiv:2605.02449, 2026.

[5] "Identification of IoT devices using neural networks and passive DNS traffic," 2025, doi: 10.3233/ATDE250715.

[6] "From flows to functions: Macroscopic behavioral fingerprinting of IoT devices via network services," arXiv:2512.16348, 2025.

[7] "SecuNet-4D: A comprehensive framework for distributed SDN security and resilience," *Sci. Rep.*, 2025, doi: 10.1038/s41598-025-98649-x.

[8] "GeckoNet: Self-healing SDN framework," ResearchGate, 2024.

[9] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[10] "Trackly: A unified platform for user behavior analytics and real-time rule-based anomaly detection," arXiv:2601.22800, 2026.

[11] "Fingerprint merging and risk-level evaluation for network anomaly detection," U.S. Patent 10 320 825.

[12] "Autonomous threat detection and response in cloud security: A survey of AI-driven strategies," arXiv:2601.03303, 2026.

[13] "Self-healing AI-driven networks for automated cyber threat detection and recovery," ResearchGate, 2025.

[14] "Towards security automation in software-defined networks," *Comput. Commun.*, 2021.